

Teaching Authentic Programming Applications to Novices: Purpose-first Tutorials in a General Education Computing Course

Mehmet Arif Demirtaş
University of Illinois
Urbana, Illinois, USA
mad16@illinois.edu

Claire Zheng
University of Illinois
Urbana, Illinois, USA
clairez5@illinois.edu

Kathryn Cunningham
University of Illinois
Urbana, Illinois, USA
katcun@illinois.edu

Abstract

Building computer science instruction for everyone requires understanding and supporting learner motivations beyond software development. Some learners value understanding the applications of code more than writing code from scratch (e.g., end-user programmers or conversational programmers). However, introductory programming instruction often emphasizes fundamental code-writing skills without clear applications. Our goal is to provide an overview of authentic computing applications to students from a variety of disciplines, including those who may not be interested in code-writing skills or may not have prior experience in programming. To this end, we designed and tested three interactive tutorials in the lab sessions of an introductory computing course for non-CS majors. Instead of teaching programming from scratch, these purpose-first tutorials highlight the purpose and basics of authentic programming libraries and tools, such as SQL queries, web-scraping libraries in Python, and HTTP requests. They are short (<1 hour), require minimal prior knowledge, and can be easily included in existing courses. The learning objectives of our activities are to help students understand why and when these tools are useful and to encourage them to explore these tools in problems that they might encounter in their disciplines. We observed that the activities were beneficial in improving confidence and providing a conceptual understanding of several tools. We present lessons learned to inform the design of similar activities for teaching computing as general knowledge to diverse audiences.

CCS Concepts

• **Social and professional topics** → **Computing literacy.**

Keywords

computing as general knowledge, purpose-first programming, non-CS majors, computational literacy, breadth-first computing

ACM Reference Format:

Mehmet Arif Demirtaş, Claire Zheng, and Kathryn Cunningham. 2026. Teaching Authentic Programming Applications to Novices: Purpose-first Tutorials in a General Education Computing Course. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1 (SIGCSE TS 2026)*, February 18–21, 2026, St. Louis, MO, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770762.3772574>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCSE TS 2026, St. Louis, MO, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2256-1/26/02

<https://doi.org/10.1145/3770762.3772574>

1 Introduction

Computing is valuable to people besides just software developers. Artists [8], managers [9], scientists [30], and analysts [22, 31] use computing and programming in ways that differ both from each other and from the conventional learning goals in CS courses. These learners may be interested in learning goals of end-user programmers, who focus on finding and modifying relevant code snippets rather than writing from scratch [23], and conversational programmers, who focus on understanding code conceptually [7]. These learning goals are common among non-CS majors [18]. However, most introductory CS instruction centers *writing code* as a main learning goal [5, 36], which is less motivating for non-CS majors [2].

There are several approaches to support the learning goals of non-CS majors. For instance, contextualized computing courses incorporate programming into other disciplines (e.g., CS+X [19]). While these courses have produced successful outcomes [25], they require extensive planning and collaboration to integrate computing into each specific discipline authentically [4]. A complementary approach is to present computing as a general education topic [1, 16]. These courses provide a high-level overview of computing, alongside multiple example applications. Their goal is to prepare students for future learning in the context of their own fields.

Although general education computing courses provide a valuable entry point, these courses often include activities that use block-based languages or highly scaffolded task-specific programming languages. However, learners may perceive these tools as inauthentic [17, 35], reducing their motivation. Therefore, there are instructional design opportunities for teaching applications of programming combined with authentic languages and tools.

In this experience report, we address this gap with three **interactive, purpose-first tutorials** that showcase goals that can be achieved with authentic programming languages and tools¹. We focus on the use cases of either a domain-specific language (e.g. SQL) or a Python library (e.g. 'BeautifulSoup' for web scraping, 'requests' for communicating with APIs). Collectively, we refer to these as **computational tools**. The tutorials are short (<1 hour), do not require prior programming knowledge, and can be deployed in a variety of courses. We designed these activities drawing on principles from prior work, and implemented them as interactive e-books on the Runestone platform [13]. These interactive tutorials are deployed in the lab sections of a "breadth-first" [10] introductory course for non-CS majors that provides an overview of the big ideas in computing, similar to AP CS Principles [27], but without a programming component. Inspired by design-based research processes, we continuously iterated on the design of the activities based on our observations and feedback from previous lab sessions.

¹Tutorials: <https://github.com/marifdemirtas/SIGCSE2025-PurposeFirstTutorials>

Through these activities, we explore the following question: **Can purpose-first activities support non-CS major learning goals in a general education computing course?** In our course, we narrowed non-CS major learning goals to two main objectives: (1) Students will be able to evaluate the value of computational tools and feel confident that they can use or learn more about these tools, and (2) Students will be able to obtain a conceptual understanding of potential applications and challenges around computational tools. Our findings provide promising evidence for both objectives and inform the design of classroom activities to support the goals of computing students beyond code-writing.

2 Related Works

2.1 Goals and Challenges of Non-CS Majors

Non-CS majors are a significant audience of computing courses, and their enrollment is increasing even faster than that of majors [6]. Why are so many non-CS majors taking computing courses? Recent work has shown that they are not looking for a path to software development careers. In a large survey of non-CS majors in computer science courses, less than four percent wanted to become professional programmers. On the other hand, only 22% believed they would not engage with programming in their future. Most non-CS majors believed they would either be writing programs in order to solve problems in their primary field (known as “end-user programming” [23]) (41% of respondents) or communicating with technical coworkers but not programming often themselves (known as “conversational programming” [7]) (34% of respondents).

In general, non-CS majors start studying computing later: non-CS majors in computing courses tend to be “latecomers,” who take their first college-level computing course in the latter half of their undergraduate degree [32]. This timeline means that non-CS majors simply have less time to learn computing. This is particularly true for conversational programmers: a large survey of non-CS majors with conversational programming goals found that 60% intended to take at most two computing courses [18]. Non-CS majors tend to have less computing experience before college [32] and also exhibit lower self-efficacy in computing than CS majors [26], which may harm their learning further. In addition, learners with end-user or conversational programming goals often express discontent with the way programming is currently taught, saying it is too focused on syntax rather than conceptual ideas and applications [11, 34].

2.2 General Education Computing Approaches

Many introductory computing courses for non-CS majors are similar to CS1 courses, with two main adjustments: (1) They make programming content easier by decreasing the pace of content and/or using a “beginner-friendly” programming language. (2) They include a broader focus beyond programming, often by emphasizing general computational thinking principles like abstraction and algorithms, and/or covering additional computing topics. For example, in the Beauty and Joy of Computing curriculum, non-CS majors program in Snap! and Python, perform basic time complexity calculations, and critique current technologies [14]. While such courses provide the additional support non-CS majors need, they typically emphasize programming skill development, minimizing the amount of time learners spend learning about applications of computing.

Contextualized computing, another approach designed for non-CS majors, teaches computing within a specific field of application. For instance, Media Computation teaches programming ideas in the context of editing images and sound [15]. While MediaComp is not ‘authentic’ as it does not use the same tools with actual media editors, it presents a plausible application of programming [17]. By contrast, the Program for Computing in the Arts and Sciences [3] offers a variety of courses that integrate computing and a particular topic, aimed at preparing non-CS majors to use computing towards their own interests and goals like data analysis for science or creative expression. Related is the CS+X approach, which teaches both computing outcomes and outcomes from another field, as in Harvey Mudd’s “BioComp” course designed for undergraduate biology students [25]. These integrated courses emphasize what computing can be used *for*, meeting the goals of non-CS majors. However, integrating computing into the context of each discipline is time-consuming, which limits the scalability of this approach and restricts the coverage of diverse computing applications.

3 Course Context

We deployed our activities in an undergraduate-level introduction to computing course, cross-listed in the information school and computer science department of a large public R1 institution in North America. This course is explicitly designed to *not* develop programming skills; instead, the goal of the course is “a broad introduction to the nature, capabilities, and limitations of computing.” The first half of the course provides an overview of core concepts in computing, including binary numbers, operating systems, time complexity, and programming languages. The second half focuses on applications, including databases, the Internet, and computer networks. Each week, students attend three hour-long lectures with the instructor and a 90-minute lab session with a teaching assistant. These lab sessions, in which we deployed our activities, provide hands-on experience in the domains covered in the lectures. There are five lab sections and each section varies between 8–40 students.

Demographics. The course serves students from a diverse set of majors, primarily in the liberal arts. The most common majors among the 116 students enrolled in Spring 2025 were Advertising (16%), Finance (8%), and Brain and Cognitive Science (8%), but 37 total majors were represented, from Political Science to Creative Writing to Nutrition. 87% of students took the course as a requirement of their Informatics minor. Mirroring other populations of non-CS majors in computing courses [32], the students tended to be later in their degree, had balanced gender dynamics, and had higher representation of underrepresented minorities in computing compared to the CS major. 55% were seniors, 26% were juniors, 18% were sophomores, and 1% were freshmen. 52% were female, and 48% were male. 34% were White, 32% were Asian, 12% were Hispanic, 10% were Black, 2% were multi-racial, and 10% were international.

On a pre-course survey, the majority of students (56%) said that they had conversational programming goals [7], i.e., they wanted to “speak the programmer’s language” but not write code often. 31% expressed that they had end-user programming goals, i.e., they planned to write task-specific programs in a non-software development job [23], and 13% believed that they would not engage with programming in their career. However, a majority of students had

some form of prior programming experience: 21% of students had not taken any programming courses before, while 22% had taken the CS1 for majors and 39% had taken a CS1 course for non-majors.

4 Activities

We designed three interactive tutorials for teaching students a computational tool (i.e., a domain-specific language or a Python library for a concrete application domain). We have two primary goals with these activities:

- **Goal 1: Students will be able to see computational tools in a demystified way, so that they will be open to learning more about these tools and using them to address problems relevant to their interests.** We consider this goal to be associated with **affective outcomes**.
- **Goal 2: Students will be able to recognize and discuss the application areas and challenges of a computational tool.** We consider this objective to be associated with **cognitive outcomes**.

These goals are informed by prior contextualized CS courses (e.g. MediaComp [15]). We focus on getting students to be interested and confident in computational tools with goal 1, and preparing them for future learning (e.g. teaching appropriate terms for a search query) with goal 2. In this section, we summarize the key principles of our design and an iterative improvement process, inspired by design-based research (DBR) approaches in prior work [13].

4.1 Design Principles

Our activities are informed by two main frameworks: we design the tutorials according to purpose-first programming [9], and we scope our content using the theory of a robust API knowledge [33].

4.1.1 Purpose-first programming. Purpose-first programming (PFP) is a set of principles for designing programming instruction that addresses common pain points faced by non-CS majors, specifically conversational programmers [9]: instruction should be brief, should prioritize the purpose of code over syntax and implementation logic, and should present examples of realistic problems that can be solved with programming. Cunningham et al. [9] designed a PFP tutorial around web scraping. Learners found the tutorial more motivating than traditional instruction, and were able to complete scaffolded tasks using web scraping with no prior knowledge.

A PFP tutorial starts with an authentic, contextual example that uses a computational tool (e.g. managing a database of movies, Figure 1(a)). This example is broken into *programming plans*, which are shorter code snippets that represent common patterns in programs that use that tool (e.g., inserting a record into a table, Figure 1(b)). These plans also include annotations that mark the parts of a plan that need to be changed in a different problem (e.g. different values to be inserted into a table). After learning about all the plans, students complete integrative exercises in which they recall, assemble, and modify multiple plans (Figure 1(c)).

The PFP structure combines multiple best practices for instructional design: A program with programming plans is similar to a worked example with subgoal labels, which supports students

by directing their attention to key concepts and keeping the cognitive load low, and the practice opportunities interleaved with instruction reinforce their learning [9].

4.1.2 Theory of API Knowledge. We draw on the theory of robust API knowledge to determine the scope of our activities [33]. Thayer et al. [33] examine programming with APIs (application programming interfaces), including “libraries, frameworks, web services, and other reusable code.” Their theory defines the knowledge necessary for not only writing code with an API, but also searching, understanding, and modifying code written with that API, which we find relevant for end-user and conversational programmer goals. Thayer et al. [33] propose three components for this knowledge:

The first component is the knowledge of **domain concepts**, which are key ideas and terminology about the API (e.g. ‘table’ and ‘query’ for SQL). In our design, we identified these concepts with the course instructor (third author), and provided direct instruction on the concepts during the lectures and at the start of each lab. The second component is the knowledge of **usage patterns**, which are code patterns that show ways of implementing a program using the API (e.g. common SQL queries). In our design, we structured our tutorials around a set of usage patterns. These patterns were identified in collaboration with the course instructor, using a tool that supports the process of finding and annotating common code patterns in a domain in the form of programming plans [20]. These patterns were presented in the tutorials as programming plans that make up complex programs. The final component is the knowledge of **execution facts**, which are mental models about a program (e.g. how a SQL query may return a view of the data). In our design, students would learn and reinforce these models in the interactive exercises where they would run code or guess the output.

4.2 Classroom Deployment

We designed and deployed three tutorial activities in the lab sessions on weeks 8, 9, and 10 of the course. We covered three computational tools: **SQL** in lab 1, for manipulating and analyzing data with relational databases; **BeautifulSoup library in Python** in lab 2, for scraping data from static webpages; and **requests library in Python** in lab 3, for communicating with APIs using JSON. These tools were chosen to align with the course content, which covers databases, HTML and markup languages, and client-server architectures. Lab 2 materials (BeautifulSoup) were adapted from the third author’s previous work on PFP [9], to test existing resources in a classroom in addition to generalizing PFP to two new libraries.

Each lab started with a short (~10 min) lecture by the TAs, recapping domain concepts. Worksheets were distributed, and included login information for the online tutorial along with a set of questions. Similar worksheets were utilized in the regular lab sessions of the course, which were also structured around an online activity and a set of related questions. While regular labs in the earlier weeks of the course were collaborative, we asked students to log in to our online tutorials individually to track log data, although discussion with peers was still encouraged. Certain questions on the worksheets explicitly directed students to the online tutorial and vice versa, so students would go through both simultaneously. Rich observation notes from each session and feedback from course TAs informed our design iterations.

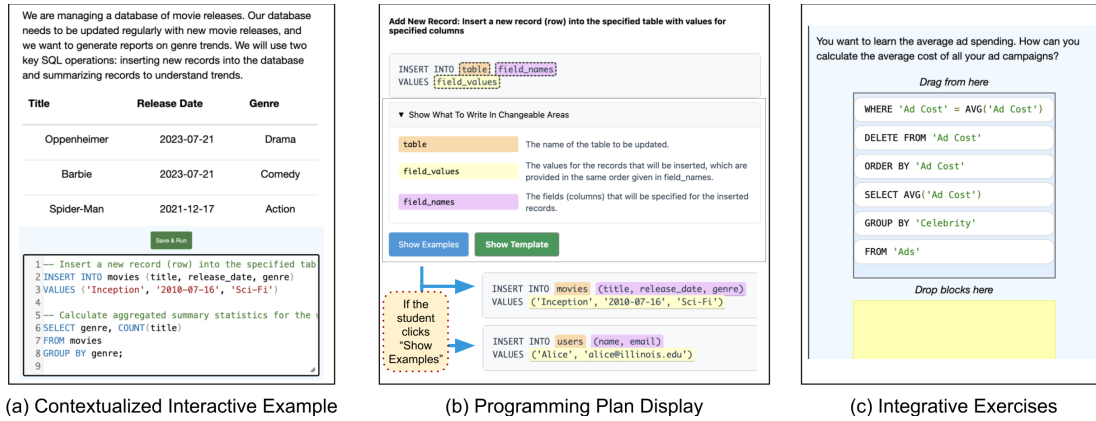


Figure 1: Three types of pages students go through in a purpose-first tutorial.

4.3 Initial Tutorial Design

Our tutorials were similar to previous PFP materials [9] and other ebooks hosted on the Runestone platform [13]. Each tutorial includes 2-3 worked examples (10-20 lines of code each). Each example includes a problem description providing context, and an interactive code environment where students can run and modify the code and see the output. This example is broken into multiple **programming plans** (1-2 lines of code) (see Figure 1). In lab 1, each plan also had a reflection activity: students were asked to write how the output changes as the plan is modified.

Each plan has a separate page that explains the purpose of the plan and contains short formative exercises. These exercises include True/False, multiple choice, and fill-in-the-blank questions, clickable code problems where students see a problem description and click on what to change to solve the problem with that plan, and Parsons problems. These problems encourage engagement, as students get unlimited attempts and immediate feedback. Some problem types, such as Parsons problems, prompted students to reset and retry the same problem to provide retrieval practice.

Finally, students completed integrative exercises that covered all plans: code reading, Parsons problems with distractors, scaffolded code writing, and explaining in Plain English (EiPE) [28]. These exercises (except EiPE) also included immediate feedback. They also completed the accompanying worksheet, which included questions about the examples in the tutorial. Each worksheet question referred to a particular section of the tutorial. The last question on the worksheet was open-ended, asking students to propose a use case for the tool that is relevant to their major or interests. Aside from the activities, students responded to optional survey questions on their experience, which we analyze in Section 5.1.

4.4 Design Iterations

Changes from Lab 1 to Lab 2. We used the set of patterns from the existing web scraping PFP tutorial [9]. Instead of designing from scratch, we used their template and modified the details of the contextualized examples (e.g. names, webpages) to fit our institution. Since we used an existing tutorial, there was no explicit direction to direct them to the worksheet. Instead, we wrote worksheet questions on the topics explored in the tutorial, but allowed students to

switch between them at their own pace. This tutorial also included more activities compared to the previous week. Also, this tutorial did not have interactive programming plan displays from Figure 1. Instead, students saw all possible values in a table on the plan page.

While discussion was encouraged in lab 1, we observed that students did not discuss the worksheets with their partners as they were completing tutorials at different paces. TAs also requested more collaborative activities. We addressed this by adding worksheet questions that explicitly ask students to stop and discuss certain concepts with their partners. We also added a question asking about any challenges students expect to encounter if they were using web scraping for an application proposed by their partners.

Changes from Lab 2 to Lab 3. While collaboration was increased from lab 1 to lab 2, TAs mentioned that students were not interacting at the same level as in the previous lab sessions in the course. To address this, we provided a single activity login to each pair of students, asking them to go through the tutorial together.

We designed this tutorial with fewer activities to be completed faster, in response to TA feedback on students getting demotivated by spending too much time. As this week's activity was designed from scratch, we also brought back two features from lab 1: interactive programming plan displays (Figure 1) and explicit direction between the tutorial and the worksheet questions. We removed the reflection activities from lab 1 that asked students to explain how changing the plan would affect the output, after observing students getting distracted by open-ended reflection prompts. Finally, we modified the wording on the worksheet question asking for challenges. Instead of asking for challenges of APIs, we asked students to compare web scraping and APIs and explain why one could be better suited for their use cases.

5 Outcomes

5.1 Goal 1: Affective outcomes

How was the student perception of the activities? Students expressed positive sentiments after each lab activity on an exit survey adapted from PSSUQ usability scale [24] (Figure 2). We gathered reactions to an additional statement after all labs were completed: "I would consider using similar tutorials to learn new tools." The average rating

was 4.77 ($SD=1.46$, $N = 111$) on a 7-point scale, indicating moderate to high agreement. We also note that students who had taken two or more courses had significantly lower ratings (3.98 , $SD = 1.41$, $N = 31$), suggesting that the activity was perceived to be more helpful by students with less prior experience. We also conducted interviews with 3 students who volunteered to provide feedback, which we discuss in Section 5.3.

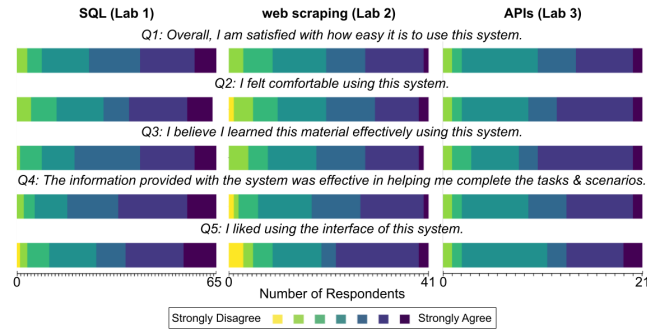


Figure 2: Distribution of responses to five usability items on a 7-point scale for each post-lab survey.

Did students feel more interested in using the computational tool after our lab sections? Before and after each lab, students rated a statement on their interest in the tool on a 7-point Likert scale. We compared their pre- and post-survey responses with Wilcoxon signed-rank tests. We did not observe any significant differences between pre- and post-survey responses for any lab (Q1 in Fig 3).

Did students feel more confident in using the computational tool after our lab sections? Similarly, students rated a statement on how confident they felt about using each tool before and after the session. We observed that students reported significantly higher confidence after the sessions in all three labs ($p < .05$) (Q2 in Fig 3).

Discussion. Our students liked learning with the tutorial, reported high feeling of learning scores (Q3 in Figure 2), and were open to learning new technologies with PFP. They also became more confident that they could use computational tools for their own goals after our tutorials, which is promising for improving self-efficacy with brief activities. Although we did not capture a significant increase in their interest in using these tools, students had already reported high interest before the activities. This echoes some studies on contextualized CS education, where students did not need the context to recognize the value of the course topics [37].

5.2 Goal 2: Cognitive outcomes

We measured cognitive outcomes that align with non-CS major learning goals through open-ended submissions for two questions.

Could students describe potential use cases for computational tools? In each lab, we included an open-ended question on the worksheets on describing use cases of the tool: In lab 1, "one [SQL] query that you might use and what you might want to achieve", in lab 2, "one task that would require you to collect information from a webpage" and in lab 3, "one task that would require you to interact with an API". Students turned in individual responses for lab 1, but could work in pairs in labs 2 and 3, resulting in fewer submissions overall.

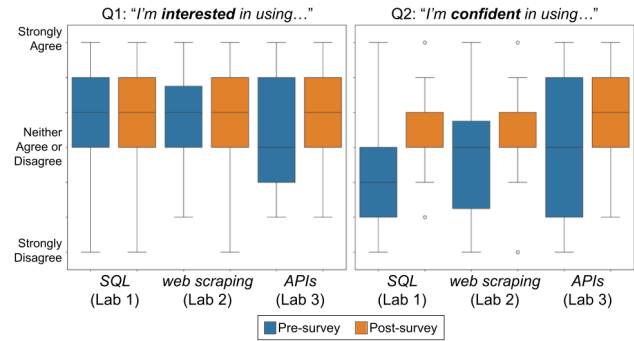


Figure 3: Student ratings on how interested (Q1) and how confident (Q2) they are in each tool before and after the lab ($N = 42$ for labs 1 and 2, $N = 20$ for lab 3)

We graded these open-ended submissions using a 5-point rubric: (0) **no response**, (1) **inaccurate** or factually incorrect response, (2) **vague** response that does not explain why the tool is appropriate or useful for that application, (3) **generic** response that repeats an example from the material, and (4) **novel** response that forms a connection between the tool and another area (e.g. student's major).² The rubric was iteratively developed as the first and second authors rated three sets of 30 randomly sampled responses until a Krippendorff's Alpha of .84 was achieved, indicating reasonable agreement. As seen in Figure 4, a majority of students were able to recognize use cases of these tools, and more than half were able to identify a novel use case not explicitly covered in the course.

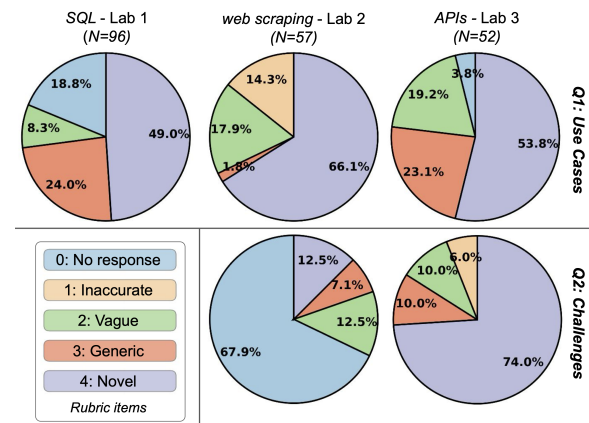


Figure 4: Distribution of student grades on questions asking for use cases (top) and challenges with tools (bottom)

Did students understand the challenges and/or tradeoffs of computational tools? In addition to applications, we wanted to evaluate if students could anticipate the potential challenges with these tools. In lab 2, we asked them to "explain a challenge you would encounter when completing [the task from the previous question]". We observed that this phrasing was confusing to some students, and we

²Worksheets: <https://github.com/marifdemirtas/SIGCSE2025-PurposeFirstTutorials>.

framed our next question around recognizing tradeoffs rather than inherent challenges. In lab 3, we asked them to discuss whether APIs and web scraping are similar and asked, "For that task, are there any advantages to using one approach over the other?"

We analyzed their responses using the same 5-point rubric, and the first and second authors rated two sets of 20 random responses until achieving a Krippendorff's Alpha of .93, indicating reasonable agreement. Figure 4 shows that students struggled to find challenges for lab 2 (web scraping library), but they were able to identify these challenges for lab 3 when they were comparing two tools rather than considering a single tool in isolation.

Discussion. Many students were able to describe novel applications of computational tools, and most could describe a somewhat accurate use case. They could also compare two similar tools and identify tradeoffs, even though they struggled to consider the challenges of a tool without any comparison. The ability to identify use cases and recognize potential tradeoffs can be valuable to support learners as they search, find, and modify relevant examples more efficiently, which is a common way end-user programmers interact with these tools [12, 29]. Moreover, as large language models improve, there have been efforts to teach prompting as a skill [21]. A conceptual understanding of what a tool can achieve and when it can fail may prove beneficial for prompting and generating code.

5.3 Lessons Learned

We reflect on our experience to inform future implementations of purpose-first activities in general education computing classrooms.

Recognize the value of short, applied computing activities. Even an hour-long session increased confidence significantly and provided an accurate overview of a tool. Two interviewees said that they would probably use at least one of these tools in their careers. One said, even though they had heard about SQL before, this activity helped them to understand what it actually is and how it works. They said it may be useful in product management to understand "how the data is structured and where the data is coming from." Thus, we suggest that even one-off, short purpose-first activities can be helpful for non-CS major learning goals.

Leave room for exploration and collaboration. Our students appreciated the interactivity in these tutorials. In most cases, students tried something until they got it working, and preferred reviewing course slides or asking peers for help rather than the TA. Our interview participants also mentioned that they liked the interactive elements the most, especially the worked examples where they can run and see the output of the code, and Parsons problems where they can build solutions by dragging and dropping blocks. We also observed students 'tinkering' with the platform by experimenting with parameters in the worked examples, retrying questions to see the feedback for other options, and repeating Parsons problems from scratch. However, this tinkering behavior was less common in lab 2, which was the lab with the most activities. We believe that planning the workload for a session to leave room for this interactive behavior may improve the student experience.

Provide flexible accommodations for different backgrounds. While students completed labs at a similar pace in general, differences in prior knowledge between students caused minor problems at times. For instance, during a lab 3 session, one student (with

notable prior programming experience) completed the activity in less than 10 minutes and left early, leading to other students getting distracted and expressing stress that they were slow. In another example, an interviewee mentioned that their lab partner for one week was familiar with that week's tool, which caused the interviewee to repeat whatever their partner showed. However, as the students could work with different partners in each session, this was not a major problem. They mentioned that they usually liked working with a partner, as it helped them to "[quickly] double-check their answers, not question themselves as much, and not second-guess themselves". Thus, we find it important to keep a balance between prior experience levels. A potential way to address this could be making these sessions optional or asynchronous. This would support further exploration for students who want to work on the material without time pressure. However, optional activities could reduce participation if students are less motivated.

Integrate domain concepts from the tutorials in the lectures. We observed that introducing domain concepts during the lecture and reviewing them in the lab sessions was helpful to the students. Students who missed the lecture or the lab reviews would get confused by some domain concepts (asking their partners or referring to course resources). This spaced repetition of domain concepts and execution facts may also help with retention. We suggest that instructors interested in using purpose-first tutorials to either modify the activities to align better with their lectures or consider altering their lectures to address key concepts explicitly.

5.3.1 Limitations and Future Work. As our students worked in pairs after lab 1, we could not investigate differences in performance based on prior experience. In our exploratory data analysis, we did not observe any indication of a significant difference between groups of students with different prior experience in their confidence, interest, or open-ended responses. However, these analyses were not conclusive due to low sample sizes of groups ($n < 10$). Future work can explore how prior experience can shape student experience with purpose-first activities. In addition, since our surveys were optional and we used similar questions for each lab, our students showed survey fatigue, leading to decreased response rates in later weeks, even though all three labs had similar attendance (86-90% of the class). In future work, we aim to identify disciplines that can benefit from PFP tutorials and evaluate how this approach transfers to other domains and tasks.

6 Conclusion

In this work, we implemented and evaluated three purpose-first programming tutorials that can be integrated into general education computing courses. In our experience, these tutorials can support the learning goals of non-CS majors. We observed promising results for both affective and cognitive outcomes. Students were more confident in learning and using computational tools for problems relevant to their interests and indicated that they would use purpose-first tutorials to learn new tools. They were also able to describe the applications and discuss tradeoffs of a computational tool, which may be especially important for non-CS majors who want to talk about code and generate code through natural language with LLMs. We believe that our findings can inform the design of CS instruction for non-CS majors in general education contexts.

References

- [1] Azad Ali and David T Smith. 2014. Teaching an Introductory Programming Language in a General Education Course. *Journal of Information Technology Education: Innovations in Practice* 13 (2014), 057–067. doi:10.28945/1992
- [2] Murtaza Ali and Sayamindu Dasgupta. 2025. "Even Though I Went Through Everything, I Didn't Feel Like I Learned a Lot": Insights From Experiences of Non-Computer Science Students Learning to Code. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 49, 18 pages. doi:10.1145/3706598.3713624
- [3] E August, Mark J Guzdial, Elizabeth Bruch, Nick Henriksen, Amir Kamil, Richard Lewis, Annette Ostling, Sarita Schoenbeck, and M Remi Yergeau. 2021. *Report of The LSA Computing Education Task Force*. Technical Report. University of Michigan.
- [4] Valerie Barr, Carla E Brodley, Elsa L Gunter, Mark Guzdial, Ran Libeskind-Hadas, and Bill Manaris. 2023. CS+ X: Approaches, challenges, and opportunities in developing interdisciplinary computing curricula. *Proceedings of ACM CS202X. ACM, New York, NY, USA* (2023).
- [5] Brett A Becker and Thomas Fitzpatrick. 2019. What do cs1 syllabi reveal about our expectations of introductory programming students?. In *Proceedings of the 50th ACM technical symposium on computer science education*. 1011–1017.
- [6] Tracy Camp, W. Richards Adrion, Betsy Bizot, Susan Davidson, Mary Hall, Susanne Hambrusch, Ellen Walker, and Stuart Zweben. 2017. Generation CS: the growth of computer science. *ACM Inroads* 8, 2 (May 2017), 44–50. doi:10.1145/3084362
- [7] Parmit K. Chilana, Celena Alcock, Shruti Dembla, Anson Ho, Ada Hurst, Brett Armstrong, and Philip J. Guo. 2015. Perceptions of non-CS majors in intro programming: The rise of the conversational programmer. In *2015 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 251–259. doi:10.1109/VLHCC.2015.7357224
- [8] Alice Mira Chung and Philip J Guo. 2024. Perpetual Teaching Across Temporary Places: Conditions, Motivations, and Practices of Media Artists Teaching Computing Workshops. In *Proceedings of the 2024 ACM Conference on International Computing Education Research-Volume 1*. 374–388.
- [9] Kathryn Cunningham, Barbara J. Ericson, Rahul Agrawal Bejarano, and Mark Guzdial. 2021. Avoiding the Turing Tarpit: Learning Conversational Programming by Starting from Code's Purpose. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–15.
- [10] Zachary Dodds, Ran Libeskind-Hadas, Christine Alvarado, and Geoff Kuenning. 2008. Evaluating a breadth-first cs 1 for scientists. *SIGCSE Bull.* 40, 1 (March 2008), 266–270. doi:10.1145/1352322.1352229
- [11] Brian Dorn and Mark Guzdial. 2010. Discovering computing: perspectives of web designers. In *Proceedings of the Sixth International Workshop on Computing Education Research (Aarhus, Denmark) (ICER '10)*. Association for Computing Machinery, New York, NY, USA, 23–30. doi:10.1145/1839594.1839600
- [12] Brian Dorn, Allison Elliott Tew, and Mark Guzdial. 2007. Introductory Computing Construct Use in an End-User Programming Community. In *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2007)*. 27–32. doi:10.1109/VLHCC.2007.35
- [13] Barbara J. Ericson, Kantwon Rogers, Miranda Parker, Briana Morrison, and Mark Guzdial. 2016. Identifying Design Principles for CS Teacher Ebooks through Design-Based Research. In *Proceedings of the 2016 ACM Conference on International Computing Education Research (Melbourne, VIC, Australia) (ICER '16)*. Association for Computing Machinery, New York, NY, USA, 191–200. doi:10.1145/2960310.2960335
- [14] Dan Garcia, Brian Harvey, and Tiffany Barnes. 2015. The beauty and joy of computing. *ACM Inroads* 6, 4 (Nov. 2015), 71–79. doi:10.1145/2835184
- [15] Mark Guzdial. 2003. A Media Computation Course for Non-Majors. In *Proceedings of the 8th Annual Conference on Innovation and Technology in Computer Science Education (ITICSE '03)*. Association for Computing Machinery, New York, NY, USA, 104–108. doi:10.1145/961511.961542
- [16] Mark Guzdial and Tamara Nelson-Fromm. 2024. A Purpose-First Theory of Transfer through Programming in a General Education Computing Course. (May 2024). doi:10.1184/R1/25587501.v1
- [17] Mark Guzdial and Allison Elliott Tew. 2006. Imagineering inauthentic legitimate peripheral participation: an instructional design approach for motivating computing education. In *Proceedings of the Second International Workshop on Computing Education Research (Canterbury, United Kingdom) (ICER '06)*. Association for Computing Machinery, New York, NY, USA, 51–58. doi:10.1145/1151588.1151597
- [18] Jinyoung Hur and Kathryn Cunningham. 2024. Profiling Conversational Programmers at University: Insights into Their Motivations and Goals from a Broad Sample of Non-Majors. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 293–311. doi:10.1145/3632620.3671123
- [19] Kathleen Isenegger, Max Fowler, Daphane Hammer, Benjamin Leff, Yael Gertner, Raya Hegeman-Davis, and Leonard Pitt. 2025. Integrating a CS+Social Science Project into STEM and Non-STEM High School Courses. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. ACM, Pittsburgh PA USA, 540–546. doi:10.1145/3641554.3701818
- [20] Yoshee Jain, Mehmet Arif Demirtas, and Kathryn Irene Cunningham. 2025. PLAID: Supporting Computing Instructors to Identify Domain-Specific Programming Plans at Scale. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 52, 21 pages. doi:10.1145/3706598.3713832
- [21] Chris Kerslake, Paul Denny, David H. Smith, James Prather, Juho Leinonen, Andrew Luxton-Reilly, and Stephen MacNeil. 2024. Integrating Natural Language Prompting Tasks in Introductory Programming Courses. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference V. 1 (Virtual Event, NC, USA) (SIGCSE Virtual 2024)*. Association for Computing Machinery, New York, NY, USA, 88–94. doi:10.1145/3649165.3690125
- [22] Mary Beth Kery, Donghao Ren, Kanit Wongsuphasawat, Fred Hohman, and Kayur Patel. 2020. The Future of Notebook Programming Is Fluid. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI EA '20)*. Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/3334480.3383085
- [23] Amy J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. 2011. The state of the art in end-user software engineering. *ACM Comput. Surv.* 43, 3, Article 21 (April 2011), 44 pages. doi:10.1145/1922649.1922658
- [24] James R Lewis. 1992. Psychometric evaluation of the post-study system usability questionnaire: The PSSUQ. In *Proceedings of the human factors society annual meeting*, Vol. 36. Sage Publications USA CA: Los Angeles, CA, 1259–1260.
- [25] Ran Libeskind-Hadas and Eliot Bush. 2013. A first course in computing with applications to biology. *Briefings in Bioinformatics* 14, 5 (02 2013), 610–617. doi:10.1093/bib/bbt005 arXiv:https://academic.oup.com/bib/article-pdf/14/5/610/580191/bbt005.pdf
- [26] Antonio M. Lopez, Marguerite S. Giguette, and Lisa J. Schulte. 2006. Large dataset offers view of math and computer self-efficacy among computer science undergraduates. In *Proceedings of the 44th Annual ACM Southeast Conference (Melbourne, Florida) (ACMSE '06)*. Association for Computing Machinery, New York, NY, USA, 158–163. doi:10.1145/1185448.1185484
- [27] Chris Mayfield. 2017. Adopting CS principles in a breadth-first survey course. *J. Comput. Sci. Coll.* 32, 5 (May 2017), 163–169.
- [28] Laurie Murphy, Renée McCauley, and Sue Fitzgerald. 2012. 'Explain in plain English' questions: implications for teaching. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (Raleigh, North Carolina, USA) (SIGCSE '12)*. Association for Computing Machinery, New York, NY, USA, 385–390. doi:10.1145/2157136.2157249
- [29] Bonnie A Nardi, James R Miller, et al. 1990. *The spreadsheet interface: A basis for end user programming*. Vol. 10. Hewlett-Packard Laboratories.
- [30] Gabrielle O'Brien. 2025. How Scientists Use Large Language Models to Program. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 876, 16 pages. doi:10.1145/3706598.3713668
- [31] Eric Rawn and Sarah E. Chasins. 2025. Pagebreaks: Multi-Cell Scopes in Computational Notebooks. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 53, 16 pages. doi:10.1145/3706598.3713620
- [32] Linda J. Sax, Kathleen J. Lehman, and Christina Zavala. 2017. Examining the Enrollment Growth: Non-CS Majors in CSI Courses. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education (Seattle, Washington, USA) (SIGCSE '17)*. Association for Computing Machinery, New York, NY, USA, 513–518. doi:10.1145/3017680.3017781
- [33] Kyle Thayer, Sarah E. Chasins, and Amy J. Ko. 2021. A Theory of Robust API Knowledge. *ACM Trans. Comput. Educ.* 21, 1 (Jan. 2021), 8:1–8:32. doi:10.1145/3444945
- [34] April Y. Wang, Ryan Mitts, Philip J. Guo, and Parmit K. Chilana. 2018. Mismatch of Expectations: How Modern Learning Resources Fail Conversational Programmers. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3173574.3174085
- [35] David Weintrop and Uri Wilensky. 2015. To block or not to block, that is the question: students' perceptions of blocks-based programming. In *Proceedings of the 14th International Conference on Interaction Design and Children (Boston, Massachusetts) (IDC '15)*. Association for Computing Machinery, New York, NY, USA, 199–208. doi:10.1145/2771839.2771860
- [36] Benjamin Xie, Dastyni Loksa, Greg L Nelson, Matthew J Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li, and Amy J Ko. 2019. A theory of instruction for introductory programming skills. *Computer Science Education* 29, 2–3 (2019), 205–253.
- [37] Svetlana Yarosh and Mark Guzdial. 2007. Narrating data structures: the role of context in CS2. In *Proceedings of the Third International Workshop on Computing Education Research (Atlanta, Georgia, USA) (ICER '07)*. Association for Computing Machinery, New York, NY, USA, 87–98. doi:10.1145/1288580.1288592